

CO5 AT2 CLOUD:

R. Deepasri (192511309)

SET 2 – Problem Solving Answers

1. MapReduce Mapper Spilling to Disk

Problem: Mapper continuously writes intermediate data to disk.

Likely causes:

- Mapper memory buffer is too small.
- `io.sort.mb` is configured too low.
- Too much intermediate data is generated.
- Poor partitioning creates data imbalance.

Solutions:

- Increase the mapper sort-buffer size within available memory.
- Tune `mapreduce.task.io.sort.mb`.
- Increase the spill percentage appropriately.
- Use a **Combiner** to reduce intermediate data.
- Compress mapper output.
- Improve partitioning to distribute keys evenly.

Result: Fewer spills → less disk I/O → faster MapReduce execution.

2. Hadoop Small File Problem

Problem: Thousands/millions of small files consume excessive NameNode memory.

Architectural mistake:

- Storing many tiny files directly in HDFS.
- HDFS is designed mainly for large files.
- Each file requires metadata in the NameNode.

Solutions:

- Combine small files into larger files.
- Use **SequenceFile, Avro or Parquet**.

- Use Hive/warehouse tables with suitable partitioning.
- Use an ingestion process that batches small files before writing to HDFS.
- Use compaction periodically.

Result: Fewer files → less NameNode metadata → better performance and scalability.

3. ML Workflow Failure After Node Restart

Problem: Model training stops or produces incomplete output after node failure.

Likely causes:

- No checkpointing.
- Temporary training state exists only on failed nodes.
- YARN containers are not properly restarted.
- Intermediate results are not stored reliably.

Solutions:

- Enable **checkpointing** at regular intervals.
- Store checkpoints in HDFS/object storage rather than local disks.
- Configure YARN automatic container/task recovery.
- Use Spark fault-tolerant execution and lineage.
- Save model versions periodically.
- Restart failed tasks from the latest checkpoint instead of starting from zero.

Result: Training can resume after node failure with minimal lost work.

4. Unauthorized Access in Secure Hadoop

Problem: Authentication and authorization are configured, but users still access sensitive data.

Likely errors:

- Incorrect HDFS permissions.
- Overly broad groups or ACLs.
- Service accounts have excessive privileges.

- Hive permissions do not match HDFS permissions.
- Missing encryption/access policies.

Solutions:

- Verify **Kerberos authentication** and user identity mapping.
- Apply least-privilege **HDFS permissions and ACLs**.
- Use **Apache Ranger** for centralized authorization and auditing.
- Apply role-based access control (RBAC).
- Enable encryption zones for sensitive HDFS data.
- Review service-account privileges.
- Maintain detailed audit logs.

Result: Only authorized users and services can access sensitive datasets.

5. SAN, NAS and HDFS Version Inconsistency

Problem: Different storage systems contain different versions of the same data.

Likely causes:

- No single source of truth.
- Synchronization jobs run at different times.
- Metadata is not properly tracked.
- Updates are overwritten without version checking.
- Duplicate synchronization processes create conflicts.

Solutions:

- Establish a **single source of truth** for master data.
- Maintain metadata containing **version number, timestamp and source ID**.
- Use incremental/CDC-based synchronization.
- Apply **idempotent** ingestion so repeated transfers do not create duplicates.
- Use checksums/hashes to detect changes.
- Use conflict-resolution rules such as latest-valid-version or source-priority.
- Maintain audit logs and data lineage.

Result: Version tracking + controlled synchronization ensures consistent data across SAN, NAS and HDFS.